<u>Pre-lab 2 and Lab 2</u> Introduction to Mininet and More Traceroute

<u>Points</u>: 100 points – 5 points for progress check in lab

<u>Due</u>: Saturday 10/12/2024 at 11:59pm

<u>Deliverables and Submission</u>: 2 files

<u>Naming convention</u>: precede filenames with **your CruzID** (before '@' in your email: id@ucsc.edu):

1. <YourCruzID>-lab2.pdf
   Submit on Gradescope: The PDF with all of your solutions to the questions.
2. <YourCruzID>-topo.py
   Submit on Canvas: Submit the Mininet topology.

Required Screenshots:

Problem #	Problem #
Q1a	Q7 b
Q3 a,b	Q8 b
Q4	Q9
Q5 a,c	
Q6 a, b	Extra Credit

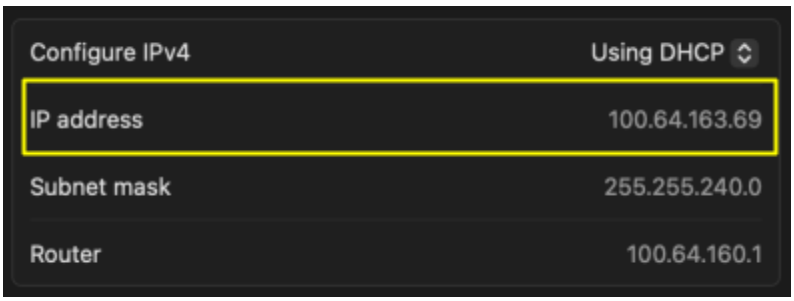
Pre-lab 2: Complete this prelab **before** attending your lab session.

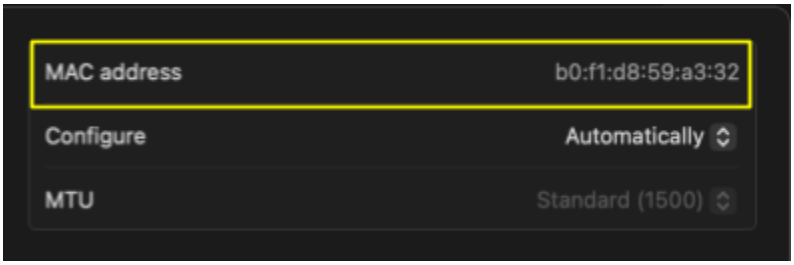
1. Read through this Wireshark tutorial. It will really help with Wireshark.
2. [5 points] **Q1**. Preliminaries: More Unix Practice
   a. [2 pts] In your own words, define Network Interface. On your laptop, look at your system settings and find your Ethernet or Wireless interface. List the interface name, MAC address, and IP Address. Attach a **screenshot** with the information highlighted or circled.

   Network Interfaces are software that allows network-specific devices to communicate with each other. It acts like the middleman between the IP layer and network-specific devices to provide the IP layer with a consistent interface to all network devices.

   Interface Name: en0
   MAC address: b0:f1:d8:59:a3:32
   IP address: 100.64.163.69

Active Services	Type	Hardware	BSD Device Name	IPv4 Addresses
AXGate VPN Client	VPN (com.axgate.AXGateVPNOSX)			
Ethernet Adapter (en4)	Ethernet	Ethernet	en4	
Ethernet Adapter (en5)	Ethernet	Ethernet	en5	
Ethernet Adapter (en6)	Ethernet	Ethernet	en6	
Thunderbolt Bridge	Ethernet	Ethernet	bridge0	
Wi-Fi	AirPort	AirPort	en0	100.64.163.69





   b. [1 pt] What is/are the commands to get details on all the processes running on the system?
   `ps` shows currently running processes
   Options such as `-A` or `aux` can be given for all processes or more detailed views.

c. [1 pt] What is the complete command to find the pid of a process named "testing"? What is the command to kill this process?

```
ps aux ¦ grep testing
Kill <PID>
```

By running `ps aux`, you can see all the processes, and `grep` will find the process called "testing". Once you have the pid, you can use the `kill` command to kill the process. (To force kill give option `-9` to kill)

d. [1 pt] What does it mean to run a process "in the background"? What is the command to run a process "testing" in the background?

A background process is executed without blocking the shell. (Only one foreground job can exist at a time, but several background jobs can be running at once since it's not blocking the shell). While the process is still running, you can continue using the terminal for other jobs/commands.

Appending the `&` at the end of the command will make the process run in the background.

Run `./testing &`

3. <u>Software Installation on your own laptop (not required):</u> You can skip this section if you plan to use the lab computers. (Just remember to save your work before you log out because it will not be saved.)
Please refer to the installation document to install Mininet on your own laptop.

Note: We will not spend lab time working on installation issues – if you do want to run Mininet on your own computer, you must complete the installation before your lab session begins.

Lab 2 Introduction to Mininet and More Traceroute

Resources for Lab 2:

- Computer Networking: A Top-down Approach Chapter 1 Section 1.4: Delay, Traceroute and Ping
- On the Mininet site, the API Reference will be an excellent resource for figuring out how to run pings or open the command prompt in between the net.start() and net.stop() lines.
- A Mininet walkthrough can be found on this page

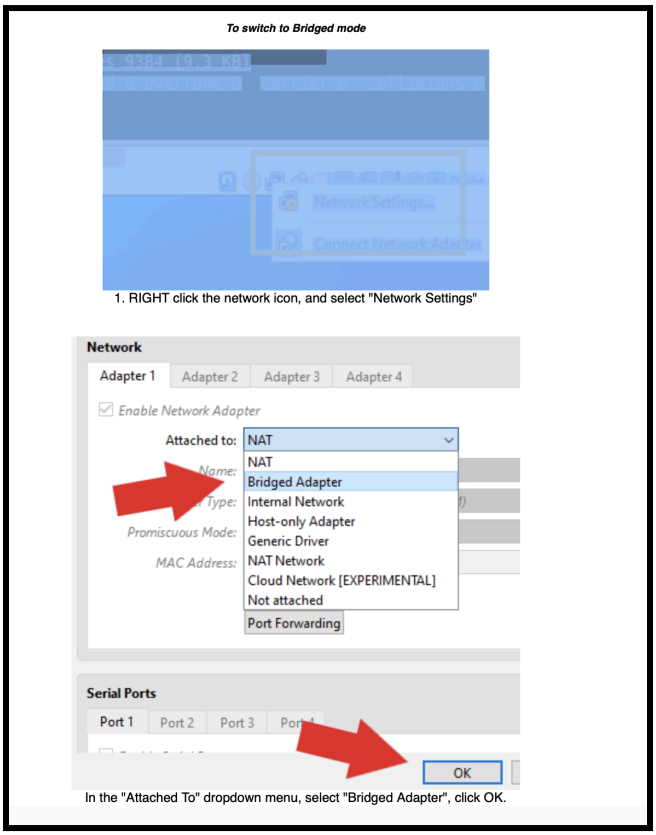
Requirements:

- All exercises MUST be run on the Mininet VM.

  Note: it is not required or expected that you install Mininet on your own laptop. You can access a fully installed VM in our labs – see Section A below.

Lab Environment:

All exercises in this lab are run on the Mininet VM. For an understanding of Bridged Mode, refer to the document: Bridge mode. To alter the mode in the Mininet VM, please follow the screenshot shown below.



[40 pts] Section A: Using Mininet

Experimenting with Mininet

A walkthrough can be found on the Mininet page. In this lab you are expected to spend some time familiarizing yourself with Mininet. First and foremost, here's some background and information on what Mininet is and why we are using it (we used to have the labs using actual routers and switches; since then we have migrated to virtualized labs).

What is Mininet? Mininet is a "network in a box" tool developed at Stanford in 2010. It is designed to allow large scale networks to be emulated in software (on a laptop). Its rise has also been dictated by the use of OpenFlow (which will be the subject of later labs). If you are interested in reading more, here is the original Mininet paper.

Why Mininet? Our previous "physical" network lab had been aging from the wear and tear of years of usage (it was donated by Cisco in 2004). Mininet is probably one of the simplest forms of network emulators. It is *free*, open source, and is widely used by the research community, as well as by universities for teaching computer networks. It allows for more interesting topologies than what can be achieved in the physical lab. Most importantly, it has enabled us to accommodate CSE 150's enrollment growth (only five years ago enrollment was around 40 students, while today it is peaking at 180). The physical lab only had 10 workstations, but over the years few were operational. So in light of the growing student base we started looking towards a solution which scales with the number of students. All students have access to a computer or the labs on campus, so instead of buying hundreds of thousands of dollars worth of specialized equipment (e.g., routers, switches), we use general-purpose computers and a VM.

How does Mininet Work? Mininet works simply by creating a virtual network. It accomplishes this task by creating host namespaces (h1, h2, etc) and connecting them through virtual interfaces. So when we run the command *ping* between the linux namespaces h1 and h2, the ping will run from h1's namespace through a virtual interface pair created for h1 and h2, before it reaches h2. If h1 and h2 are connected through a switch as shown in the Python code in the Mininet walkthrough, the ping will transit multiple virtual interface pairs. The switches that we will be using are running OpenVSwitch (OVS). Mininet will connect additional virtual interfaces between each virtual port on the switch with each connected host. The host name space allows each host to see the same file system, but operates as its own process that will run separately from each of the other host processes. The OVS version running on the Ubuntu image supports OpenFlow.

Understanding some Mininet commands:

We recommend practicing with the following Mininet commands before you start the lab exercises:

1. ***sudo mn***: will start Mininet

```
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

2. You can use ***mn -h*** or type ***help*** after you have run Mininet.

```
mininet> help

Documented commands (type help <topic>):
========================================
EOF    gterm  iperfudp  nodes        pingpair      py      switch
dpctl  help   link      noecho       pingpairfull  quit    time
dump   intfs  links     pingall      ports         sh      x
exit   iperf  net       pingallfull  px            source  xterm

You may also send a command to a node using:
  <node> command {args}
For example:
  mininet> h1 ifconfig

The interpreter automatically substitutes IP addresses
for node names when a node is the first arg, so commands
like
  mininet> h2 ping h3
should work.

Some character-oriented interactive commands require
noecho:
  mininet> noecho h2 vi foo.py
However, starting up an xterm/gterm is generally better:
  mininet> xterm h2
```

3. We can see that when we launched Mininet above, a mini-network was created.
   a. The ***net*** command shows that *h1* indicates host 1 has one network interface *eth0,* which is connected to the switch on interface *eth1* (on the output line: *h1 h1-eth0:s1-eth1)*. There is also host 2 (*h2*), switch 1 (*s1*), and controller 0 (*c0*).

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=1049>
<Host h2: h2-eth0:10.0.0.2 pid=1052>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=1057>
<Controller c0: 127.0.0.1:6633 pid=1042>
```

   b. From the **dump** command output we can also see what IP addresses have been assigned to h1 and h2. We also are given the pids of each process. Processes in Mininet are used for hosts, switches, and controllers. Mininet is composed of processes using Interprocess Communication (IPC) to emulate a network environment.
4. ***sudo mn -c***: cleans the Mininet system. **Use this if you are having errors to start Mininet**, in case some problem from a previous execution has persisted.

Building Your Own Network in Mininet:

Q2. [5 pts] **Mininet Commands**

a. What command do you use to find the interfaces of a host?
`<host-name> ifconfig`

b. What are 2 different ways to test connectivity between hosts and what is the difference between them?
`<host 1> ping <host 2>`
- Tests whether one host can reach another host and measures RTT

`<host 1> iperf -s`
`<host 2> iperf -c <host 1>`
- Measures the network bandwidth between the hosts

c. What command do you use **in Mininet** to open a terminal on a host?
`xterm <host-name>`

Q3. [15 pts] **Write your own topology script**

a. Complete the Python skeleton topology file to create the topology in Figure 1. Run a command to display connections between the devices listed below, attach a **screenshot** of the output and highlight these connections in YELLOW:

- Desktop and Laptop are connected to Switch1
- LIghts and Fridge are connected to Switch2

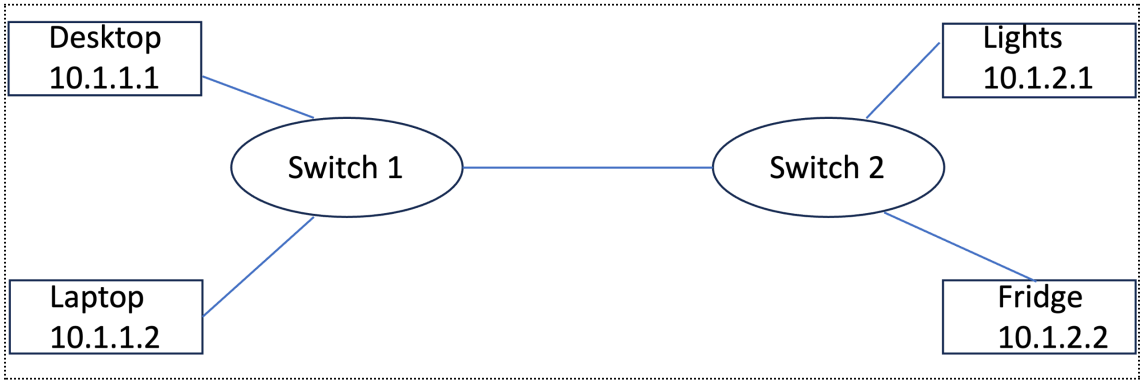


Figure1

```
mininet@mininet-vm:~$ sudo python3 topology.py
mininet> net
Desktop Desktop-eth0:switch1-eth1
Fridge Fridge-eth0:switch2-eth2
Laptop Laptop-eth0:switch1-eth2
Lights Lights-eth0:switch2-eth1
switch1 lo:  switch1-eth1:Desktop-eth0 switch1-eth2:Laptop-eth0
switch2 lo:  switch2-eth1:Lights-eth0 switch2-eth2:Fridge-eth0
c0
mininet>
```

b. Run a command that will display the IP address of ALL of the devices in Figure 1. Attach a **screenshot** of the output after running the command and highlight the IP addresses of the Laptop and Lights in the screenshot.

```
mininet> dump
<Host Desktop: Desktop-eth0:10.1.1.1 pid=19924>
<Host Fridge: Fridge-eth0:10.1.2.2 pid=19926>
<Host Laptop: Laptop-eth0:10.1.1.2 pid=19928>
<Host Lights: Lights-eth0:10.1.2.1 pid=19930>
<OVSSwitch switch1: lo:127.0.0.1,switch1-eth1:None,switch1-eth2:None pid=19935>
<OVSSwitch switch2: lo:127.0.0.1,switch2-eth1:None,switch2-eth2:None pid=19938>
<Controller c0: 127.0.0.1:6653 pid=19917>
mininet>
```

Q4. [5 pts] **Verifying connectivity**

On the Mininet command line run ***pingallfull*** and explain what you understand from the output. Attach a **screenshot** of the output.

```
mininet> pingallfull
*** Ping: testing ping reachability
Desktop -> Fridge Laptop Lights
Fridge -> Desktop Laptop Lights
Laptop -> Desktop Fridge Lights
Lights -> Desktop Fridge Laptop
*** Results:
 Desktop->Fridge: 1/1, rtt min/avg/max/mdev 247.885/247.885/247.885/0.000 ms
 Desktop->Laptop: 1/1, rtt min/avg/max/mdev 164.479/164.479/164.479/0.000 ms
 Desktop->Lights: 1/1, rtt min/avg/max/mdev 244.945/244.945/244.945/0.000 ms
 Fridge->Desktop: 1/1, rtt min/avg/max/mdev 123.335/123.335/123.335/0.000 ms
 Fridge->Laptop: 1/1, rtt min/avg/max/mdev 247.713/247.713/247.713/0.000 ms
 Fridge->Lights: 1/1, rtt min/avg/max/mdev 166.092/166.092/166.092/0.000 ms
 Laptop->Desktop: 1/1, rtt min/avg/max/mdev 86.000/86.000/86.000/0.000 ms
 Laptop->Fridge: 1/1, rtt min/avg/max/mdev 123.954/123.954/123.954/0.000 ms
 Laptop->Lights: 1/1, rtt min/avg/max/mdev 250.110/250.110/250.110/0.000 ms
 Lights->Desktop: 1/1, rtt min/avg/max/mdev 124.416/124.416/124.416/0.000 ms
 Lights->Fridge: 1/1, rtt min/avg/max/mdev 82.282/82.282/82.282/0.000 ms
 Lights->Laptop: 1/1, rtt min/avg/max/mdev 123.802/123.802/123.802/0.000 ms
mininet>
```

For each pair of hosts, mininet checks whether the hosts can successfully ping. (1/1 means one packet was sent and one packet was received). From the output, we can see that all Desktop, Fridge, Laptop, and lights are reachable from each other. All packets sent were received and all nodes were reachable from each other.

It also shows the minimum, average, max, and mean deviation of the RTT, which can help us understand the delay or latency between devices in the network.

Q5. [15 pts] **Wireshark Analysis for topology in Figure 1 and Understanding Ping**

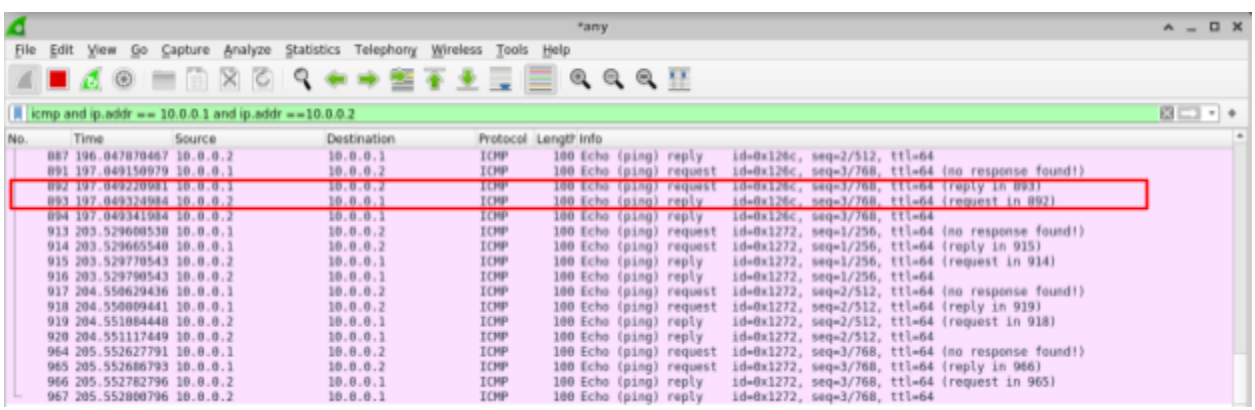
The Ping utility relies on the ICMP protocol. What is ICMP? Your textbook has a good small section summarizing ICMP.
ICMP (Internet Control Message Protocol) is the protocol used to support the internet protocol. It is used by network devices to send error messages and operational information indicating success/failure when communicating with another IP address. The ICMP differs per protocol such as TCP and UDP.

While you are still in your mininet environment, run Wireshark as "**sudo wireshark &**" and choose the "any" interface to capture packets on all interfaces [the '&' means that wireshark will run in the background].

a. Run Ping from a host to any other host using `X ping -c 3 Y` (replace X and Y with the names of the source host and destination host respectively). Use the display filter to find the ICMP packets relevant to the ping command you just ran. Attach a **screenshot** and highlight the 2 types of ICMP messages which were sent and received by Ping.

   I ran `h1 ping -c 3 h2`, where h1 = 10.0.0.1 and h2 = 10.0.0.2

   The two types of messages–ICMP Echo request and reply–are highlighted in the screenshot below

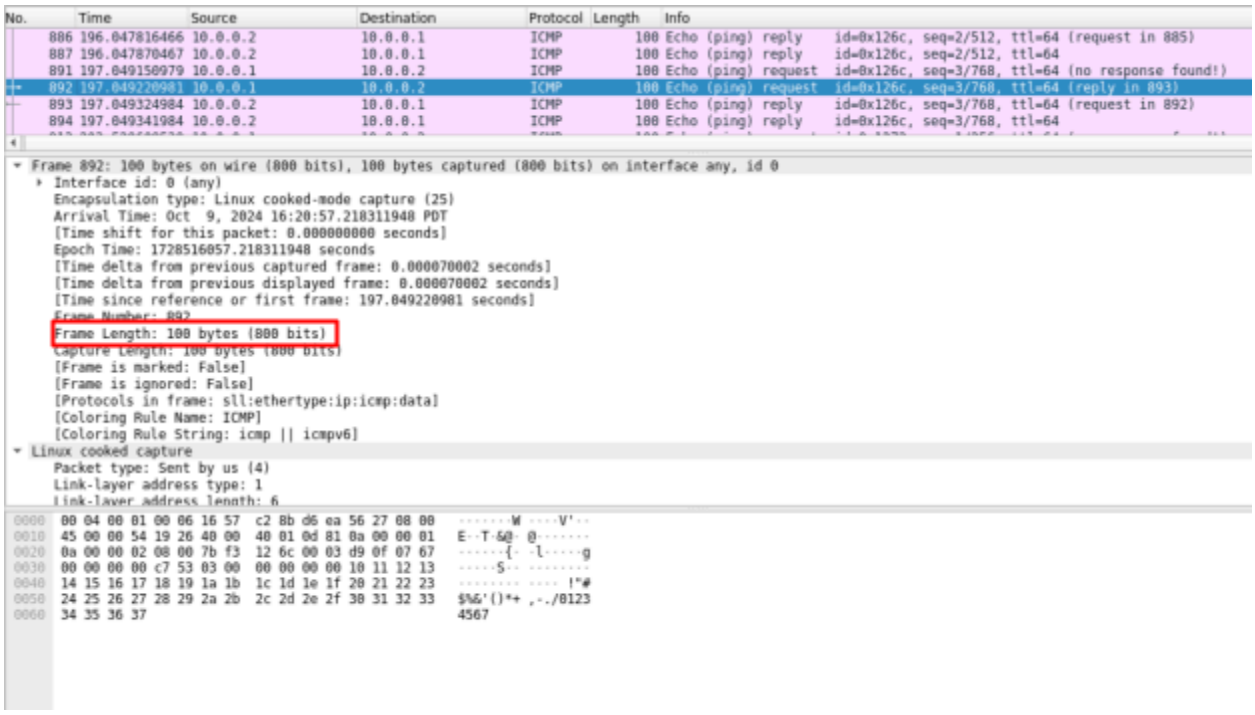


b. Describe how PING uses these two ICMP message types.

Ping sends an ICMP echo request to the target device, and if the device is within reach, it will send an ICMP echo reply message, acknowledging successful communication.

c. In Wireshark, select and drill into the details of a single ICMP ECHO request. Attach a **screenshot** showing the detailed breakdown of this packet. Circle the **total length** of the transmission frame.

Total length (frame length) = 100 Bytes
(Sorry I couldn't circle. Put a box instead so that the text wouldn't get covered)



[50 pts] Section B: Deep Dive into Traceroute Using Wireshark over the Internet

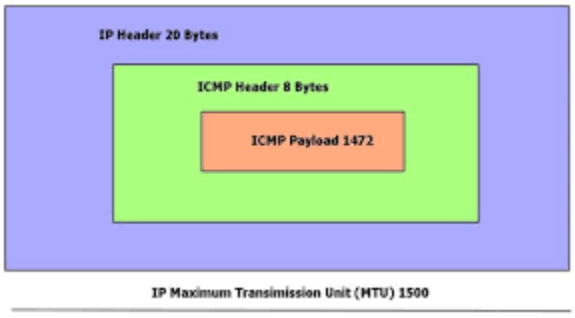
Traceroute Background:

The Traceroute program sends a series of probes from a source host to a destination. In each iteration the probes get closer to the destination until finally, in the final iteration, the destination is reached. Traceroute's iterations operate in this way:

- First iteration: one or more (usually 3) datagrams (probes) transmitted with the time-to-live (TTL) field in the IP header set to 1. This causes the probe to end at the first router, which returns an ICMP message (type 11 – **TTL-exceeded)** to the source host.
- Second iteration: one or more probes transmitted towards the same destination with a TTL value of 2. This causes the probe to end at the second router on the path to the destination, which returns an ICMP message (type 11 – **TTL-exceeded)** is returned to the source host.
- Third iteration: one or more probes transmitted towards the same destination with a TTL value of 3
- Subsequent iterations: and so on… until the final destination is reached.

Recall that when a packet is received by a router, the router decrements the TTL by 1 (actually, RFC 791 says that the router must decrement the TTL by *at least* one). Upon receiving a packet with a TTL=0, routers return an ICMP message (type 11 – **TTL-exceeded**) to the sending host.

To help visualize the packet transfer, these images show how ICMP messages are carried in an IP datagram which begins with an IP header:



IPv4 Datagram

	Bits 0–7	Bits 8–15	Bits 16–23	Bits 24–31
Header (20 bytes)	Version/IHL	Type of service	Length	
	Identification		*flags* and *offset*	
	Time To Live (TTL)	Protocol	Header Checksum	
	Source IP address			
	Destination IP address			
ICMP Header (8 bytes)	Type of message	Code	Checksum	
	Header Data			
ICMP Payload (*optional*)	Payload Data			

Traceroute Probes: Traceroute can operate by sending probes using either the UDP protocol or by using ICMP Echo Request messages. You will investigate using Wireshark to determine which method is being used in your traces.

In this manner, the host executing traceroute can learn the IP addresses of the routers on the path between itself and the destination by looking at the source IP addresses of the ICMP **TTL-exceeded** messages. The Traceroute program nicely displays that information for you in its output.

Note: Make sure that you are seeing <u>complete Traceroute output</u>:

- **What is complete Traceroute output?** The output is considered to be complete when Traceroute terminates and the final destination is reached.
- **Is it OK to see some *** in my Traceroute output?** Yes, but overall the majority of lines display a normal response.
- **What do I do if Traceroute does not complete?** Several options here: a) Make sure the VM is running in the correct mode (see Lab Environment). b) If allowed in the question, try a different destination. c) If the destination is given and your output is still not complete, consult with your TA or a tutor, attend office hours.

<u>Traceroute Exercises:</u> **All exercises are to be run on the Mininet Virtual Machine (either installed on your own computer or in the lab). Traces run from other environments cannot be graded.**

In this section, you will use Wireshark to observe and identify the iterations of outgoing Traceroute probes (increasing TTL values) and the corresponding **TTL-Expired** messages returned by routers on the path (or by final destination) in response. Pay close attention to your Traceroute output - if it doesn't complete, check the instructions above.

Q6 [15 pts] **Warmup: Search for Traceroute Probes, TTL and TTL-Exceeded**

Run Traceroute in "Bridged" mode on the lab computer in the Mininet VM or on your personal laptop with the mininet VM. Start up a Wireshark capture using the interface *eth0*. Then run this command (without any additional flags or modifications) in a VM terminal:

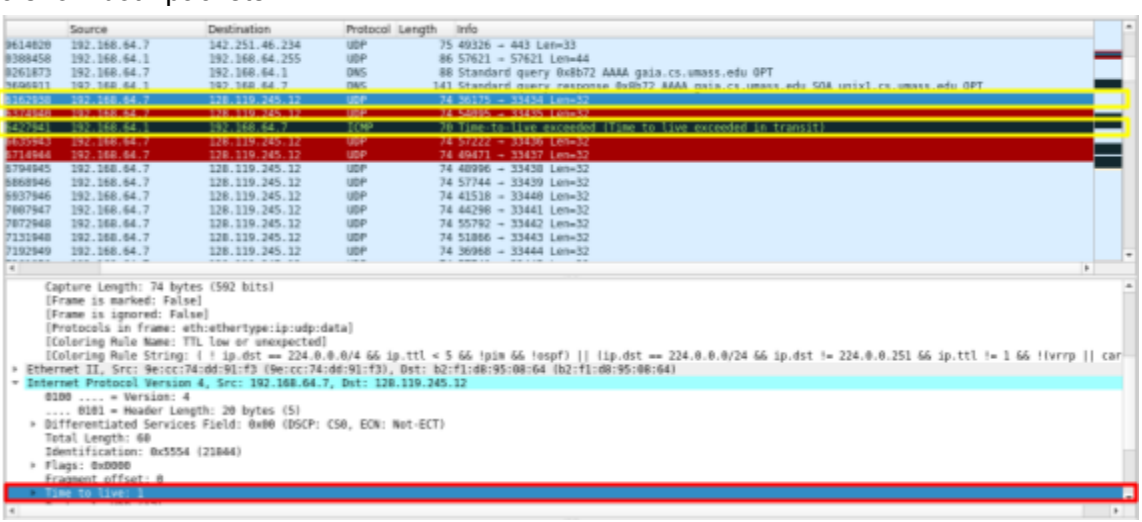
```
traceroute -q 1 gaia.cs.umass.edu
```

Stop the capture when `traceroute` is done. A complete Traceroute output needs to be captured in Wireshark for this problem. If you have trouble, follow the isudo nstructions above in **What do I do if Traceroute does not complete?** .

a. What is the meaning of the -q flag in the traceroute command?
   `-q` option specifies the number of probe packets to be sent per hop. By default it is 3.

b. Search for two related packets:
   - Packet #1: an `ICMP` **`TTL-Exceeded`** message returned by a router to the Client (the `traceroute` process running on your VM)
   - Packet #2 (Back in Time): the probe packet sent by your VM (client) that caused the **`TTL-Exceeded`** message (Packet #1) to be generated. This packet precedes Packet #1.

1. What was the value for `TTL` in the probe packet (Packet #2)? (Hint: you can find the `TTL` field if you drill into the IP header). What is the meaning of this TTL? **Include a screenshot** highlighting the **2 packets** and **the TTL of packet #2**. You may use a filter to show both packets.



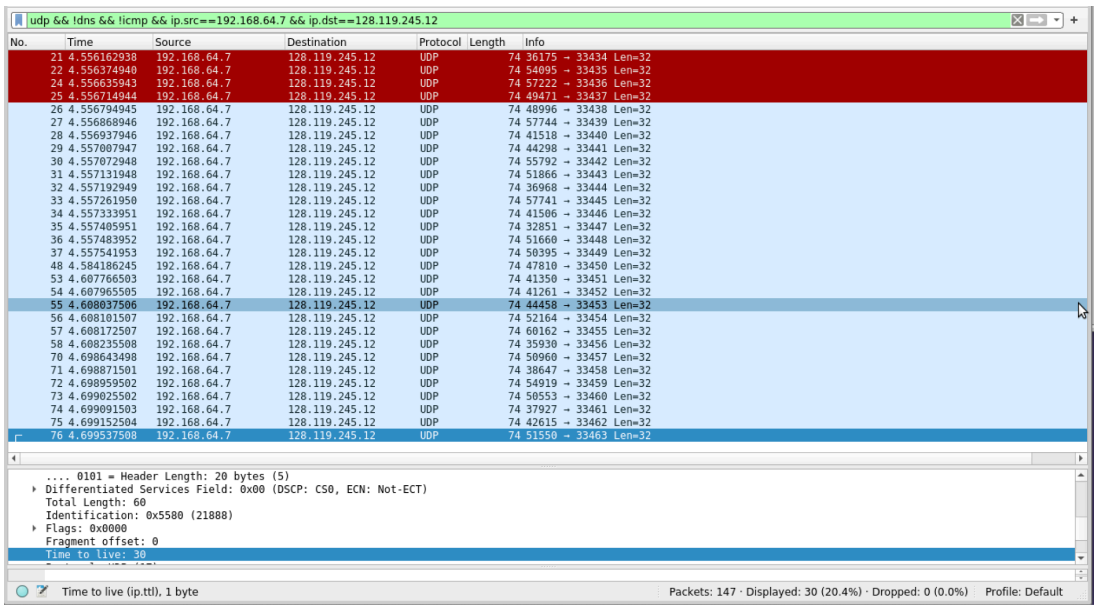
- The two packets are highlighted in the yellow box

  TTL of packet #2 was 1 (highlighted in the red box below). A TTL of 1 means that the packet can only make one hop before being discarded by the next router it encounters. The packet will be dropped at the next router.

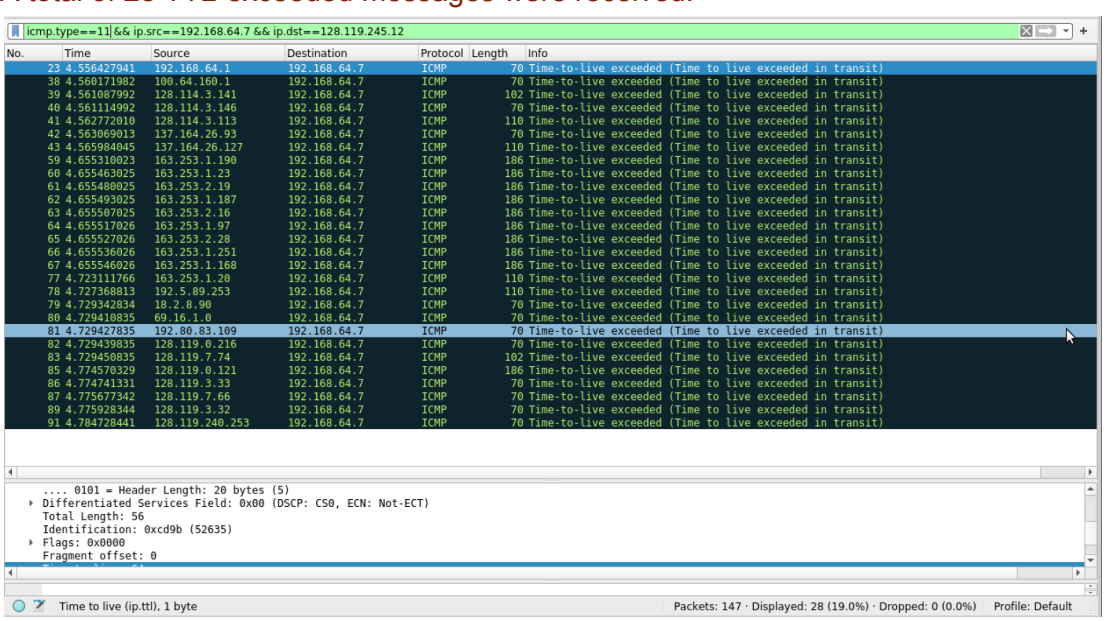
2. According to your Wireshark output, how many probes in total were transmitted? Does each generate a TTL-Exceeded message? Explain and show all of the probes in a **Screenshot** to justify your answer.

My results show **30 probes transmitted** and 28 TTL-exceeded messages. **Each probe generates a TTL-Exceeded message**. While the two numbers should be the same, the packets may have been dropped due to network traffic or congestion; the messages may have been lost on the way somewhere. Still, the number of probes and TTL-exceeded messages are close to each other in my results.

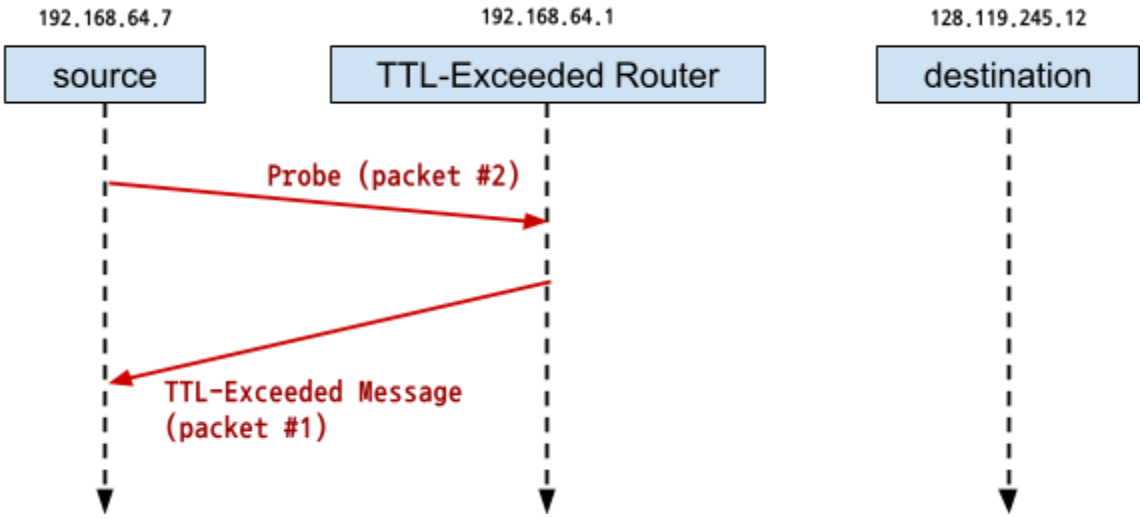
There were a total of 30 probes sent:



A total of 28 TTL-exceeded messages were received:



c. Draw a small timing diagram illustrating the transmission of the probe sent from your Client to the router, and the returning TTL-Exceeded message for the TTL iteration you have chosen from 6b. Label each entity (Source, Destination, TTL-Exceeded Router). The timing diagram can look like the protocol exchange in Figure 1.2 of the textbook.



Q7 [10 pts] **Discover the protocol for Traceroute Probes**

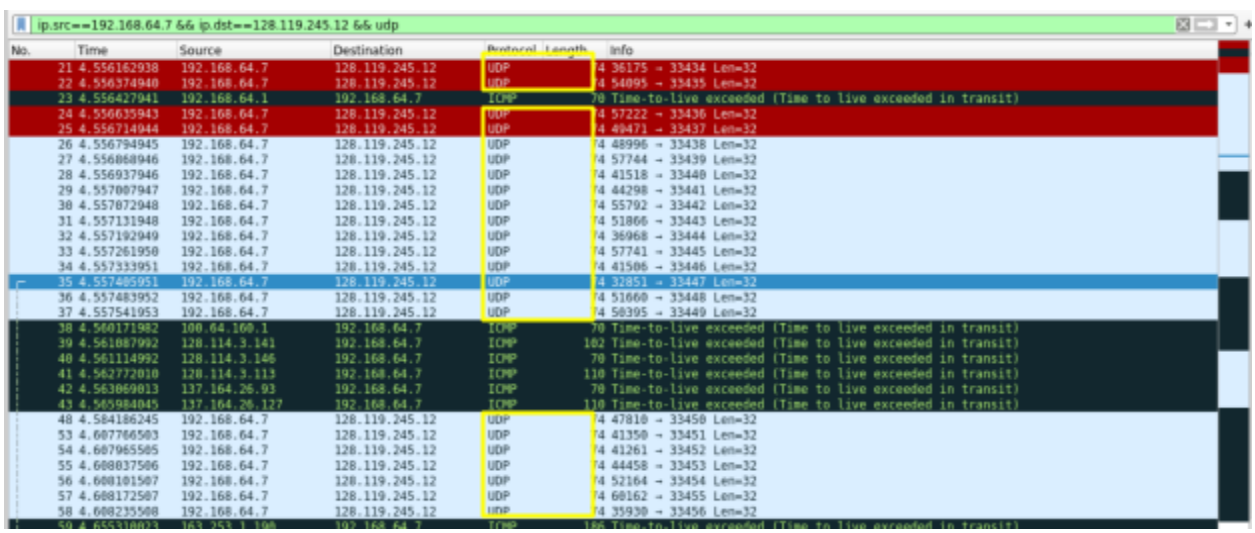
a. What 2 protocols can Traceroute use for its probes? (Review Traceroute Background above).

Traceroute can operate by sending probes using either the **UDP protocol or by using ICMP Echo Request messages.**

Continuing with the Wireshark capture completed in Question 6:

b. From the Wireshark capture, which protocol is being used to send the <u>probe</u> in Q6 (b)? **<u>Attach a screenshot</u>** and circle information that leads to your conclusion. <u>Explain</u> your reasoning.

UDP is being used to send the probe. If we check the messages that are being sent from the source, they are using the protocol UDP (highlighted in the yellow box in the screenshot). Furthermore, I checked that the UDP probes had TTL being incremented by 1 as time passed (traceroute increments its probes TTL by one until it terminates)



ip.src==192.168.64.7 && ip.dst==128.119.245.12 && udp

No.	Time	Source	Destination	Protocol	Length	Info
21	4.556162938	192.168.64.7	128.119.245.12	UDP	74	36175 → 33434 Len=32
22	4.556374940	192.168.64.7	128.119.245.12	UDP	74	54095 → 33435 Len=32
23	4.556427941	192.168.64.1	192.168.64.7	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
24	4.556635943	192.168.64.7	128.119.245.12	UDP	74	57222 → 33436 Len=32
25	4.556714944	192.168.64.7	128.119.245.12	UDP	74	49471 → 33437 Len=32
26	4.556794945	192.168.64.7	128.119.245.12	UDP	74	48996 → 33438 Len=32
27	4.556868946	192.168.64.7	128.119.245.12	UDP	74	57744 → 33439 Len=32
28	4.556937946	192.168.64.7	128.119.245.12	UDP	74	41518 → 33440 Len=32
29	4.557007947	192.168.64.7	128.119.245.12	UDP	74	44298 → 33441 Len=32
30	4.557072948	192.168.64.7	128.119.245.12	UDP	74	55792 → 33442 Len=32
31	4.557131948	192.168.64.7	128.119.245.12	UDP	74	51866 → 33443 Len=32
32	4.557192949	192.168.64.7	128.119.245.12	UDP	74	36968 → 33444 Len=32
33	4.557261950	192.168.64.7	128.119.245.12	UDP	74	57741 → 33445 Len=32
34	4.557333951	192.168.64.7	128.119.245.12	UDP	74	41506 → 33446 Len=32
35	4.557405951	192.168.64.7	128.119.245.12	UDP	74	32851 → 33447 Len=32
36	4.557483952	192.168.64.7	128.119.245.12	UDP	74	51660 → 33448 Len=32
37	4.557541953	192.168.64.7	128.119.245.12	UDP	74	50395 → 33449 Len=32
38	4.560171982	100.64.160.1	192.168.64.7	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
39	4.561087992	128.114.3.141	192.168.64.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
40	4.561114992	128.114.3.146	192.168.64.7	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
41	4.562772010	128.114.3.113	192.168.64.7	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
42	4.563069013	137.164.26.93	192.168.64.7	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
43	4.565984045	137.164.26.127	192.168.64.7	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
48	4.584186245	192.168.64.7	128.119.245.12	UDP	74	47810 → 33450 Len=32
53	4.607766503	192.168.64.7	128.119.245.12	UDP	74	41350 → 33451 Len=32
54	4.607965505	192.168.64.7	128.119.245.12	UDP	74	41261 → 33452 Len=32
55	4.608037506	192.168.64.7	128.119.245.12	UDP	74	44458 → 33453 Len=32
56	4.608101507	192.168.64.7	128.119.245.12	UDP	74	52164 → 33454 Len=32
57	4.608172507	192.168.64.7	128.119.245.12	UDP	74	60162 → 33455 Len=32
58	4.608235508	192.168.64.7	128.119.245.12	UDP	74	35930 → 33456 Len=32

Q8 [10 pts] **Switch the Protocol**

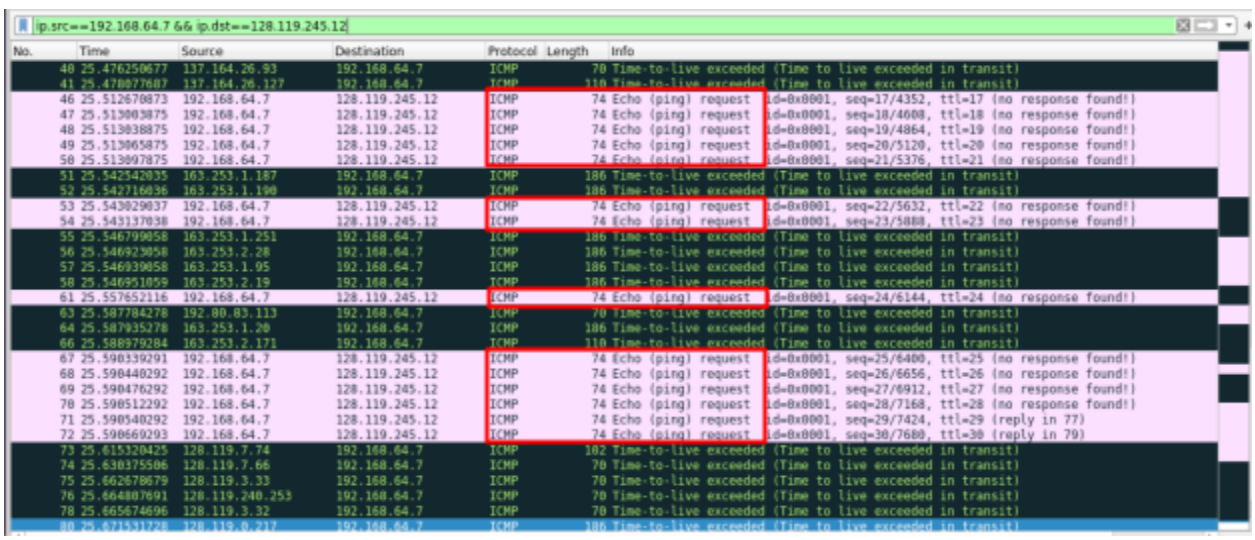
a. According to the man page in the VM, what flag would you need to use to force `traceroute` to use the ICMP protocol?

Now run `traceroute` to the same destination used in Q6, but force `traceroute` to switch the protocol it uses for its probes.

You would use the **flag** `-I` to force traceroute to use the ICMP protocol:

`Ex: traceroute -I [destination]`

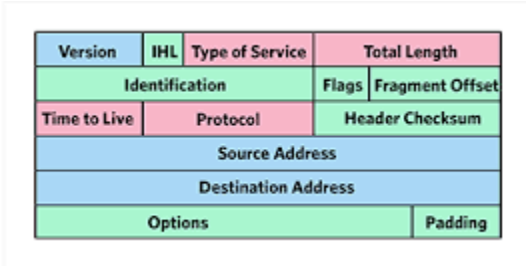
b. Use Wireshark to verify that Traceroute is using the forced protocol. **Include a screenshot** with packets <u>circled or highlighted</u> that verify Traceroute is using a different protocol for its probes. Provide an explanation.



From the packets that are highlighted in the red box in the screenshot above, it can be verified that traceroute is using the ICMP protocol for its probes instead of UDP. It is sending an Echo (ping) request, and there are TTL-Exceeded messages that correspond to those ICMP probes.

Q9 [15 pts] **Checking out the IP header (Layer 3 in protocol stack)**

Reference: Section 4.3 (The Internet Protocol (IPv4) of text book, Figure 4.16 shows the IP Packet Header



Run traceroute to a destination of your choice (make sure Traceroute completes-i.e., the destination is reached). We suggest running Traceroute with only 1 probe per hop - this will make it much easier to find a probe / response pair. Filter to show only UDP and ICMP packets. Find one of the **TTL-Exceeded** packets received by your VM and examine the IP Packet Header. To do this, expand the 'Internet Protocol Version 4' section within the packet details window.

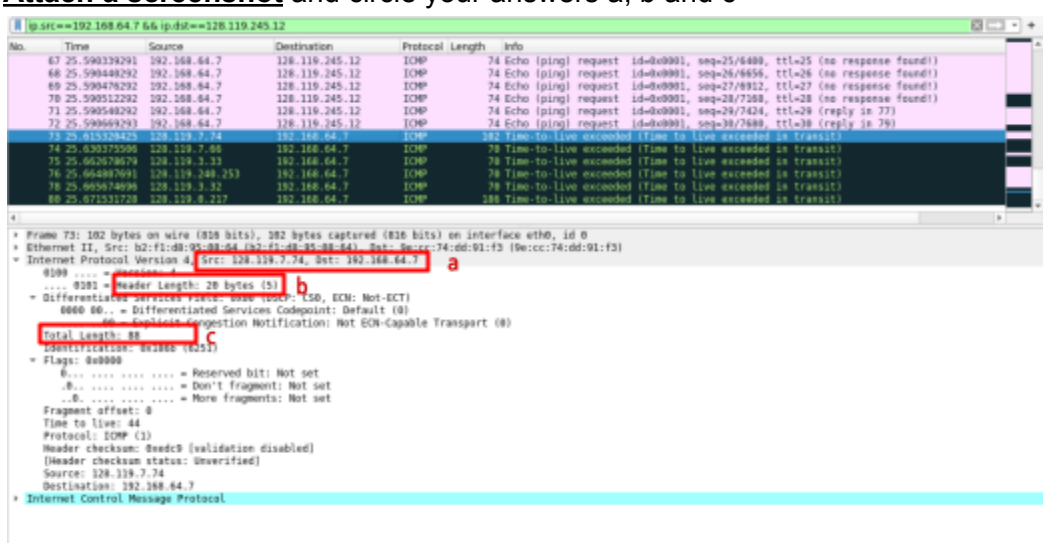
From the displayed IP header information:

a. What is the source (`Src`) and destination (`Dst`) IP address?
   Src = 128.119.7.74
   Dest = 192.168.64.7

b. What is the size (in bytes) of the IP header's Header Length field?
   20 bytes

c. How many bytes are in the payload of the IP datagram? Explain how you determined the number of payload bytes. Ask for help in the lab!
   68B (Total length (88B) - header (20B) = 68 B)

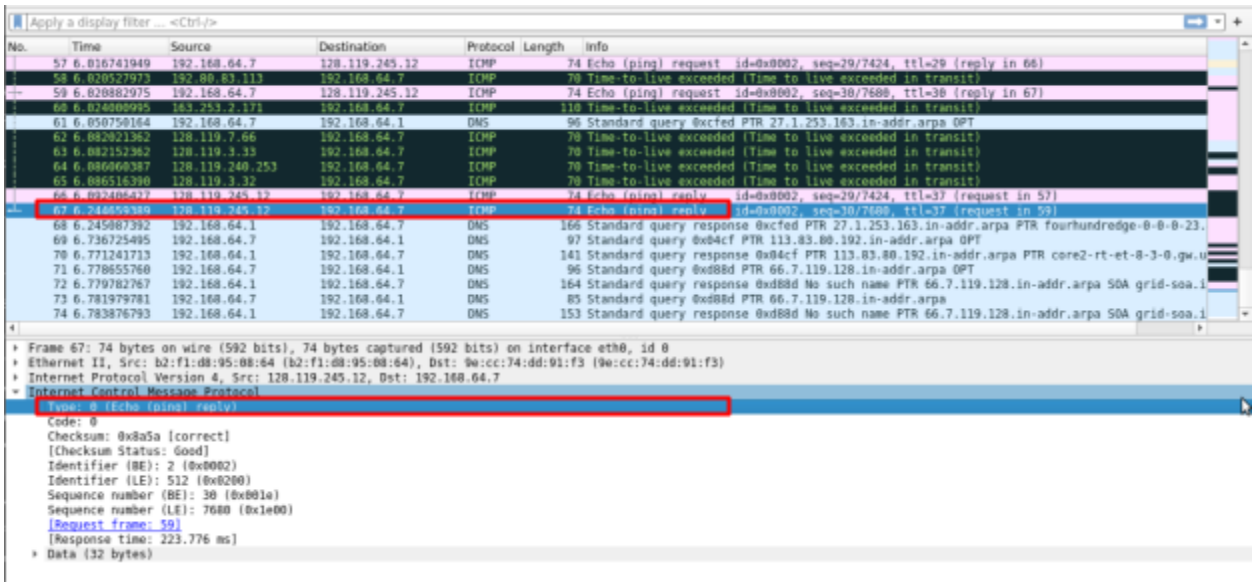
Attach a screenshot and circle your answers a, b and c



Extra Credit (optional) [5 pts] The Final Response

Think about the probes you observed in Q6. Find the response from the Server that causes Traceroute to terminate. If you do not see a final response in Wireshark, try another site that does send a response. Remember, we have no control over how hosts are configured – some respond to UDP probes and final ICMP probes, others do not.

<u>Attach a Screenshot</u> of your results in Wireshark and describe what type of packet you have found that causes Traceroute to finish. (If you cannot find the final probe, write an explanation of your expectation.)



I found the **Echo (ping) reply packet** that caused traceroute to finish. The echo reply packet indicates that the probe sent successfully reached the destination.